

UNIDAD 6

PROGRAMACIÓN ORIENTADA A OBJETOS

(INTRODUCCIÓN)

**Programación
CFGS DAW**

Adaptado y ampliado por:
José Miguel Blázquez

Autores iniciales:
Carlos Cacho, Raquel Torres, Lionel Tarazón

2023/2024

Licencia



Reconocimiento - NoComercial - CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:



Importante



Atención



Interesante

ÍNDICE DE CONTENIDO

1. Introducción a la programación orientada a objetos.....	4
2. Fundamentos de las clases.....	5
2.1 Visibilidad de los miembros de una clase.....	7
2.2 Métodos.....	8
2.3 Constructores.....	9
3. Objetos.....	10
3.1 Instanciación (creación) de un objeto.....	10
3.2 Asignación de variables de referencia a objetos.....	10
3.3 El operador punto (.).....	11
4. Constantes de clase y de objeto.....	12
5. Arrays de objetos.....	13
6. Ejemplos.....	14
6.1 Ejemplo 1.....	14
6.2 Ejemplo 2.....	16
6.3 Ejemplo 3.....	17
6.4 Ejemplo 4.....	21
6.5 Ejemplo 5.....	22
6.6 Ejemplo 6.....	22

UD06. PROGRAMACIÓN ORIENTADA A OBJETOS (I)

1. INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

La Programación Orientada a Objetos (POO) es un estilo de programación, perteneciente al paradigma imperativo, que permite enfrentarse a problemas más complejos y aplicaciones más extensas, mediante el modelado de la realidad en base al concepto de “objetos”. Este enfoque permite entender los problemas de forma más cercana a la realidad, mediante la interacción de dichos objetos entre ellos ya que uno puede funcionar de forma totalmente independiente. En la POO, se organiza el código alrededor de objetos que son instancias de clases. Los objetos representan entidades del mundo real y las interacciones entre ellas. Este enfoque facilita la modelización y resolución de problemas complejos.

 Un **objeto** es un elemento del programa que posee sus propios datos y su propio funcionamiento. Representa un concepto, similar a las “Entidades” del modelo E-R.

 Una **clase** describe un grupo de objetos que contienen una información similar (atributos) y un comportamiento común (métodos), como si fuera una plantilla.

 Antes de poder utilizar un objeto, se debe definir su clase. La clase es la definición de un tipo de objeto (la plantilla con la que se van a producir).

Al definir una clase lo que se hace es indicar cómo funciona un determinado tipo de objetos, indicando sus características y las acciones que pueden realizar. Luego, a partir de la clase, podremos crear objetos de esa clase como si generáramos objetos a partir de un molde o plantilla.

Las **propiedades fundamentales** de la POO son la siguientes:

- **Abstracción:** se trata de simplificar la complejidad del sistema u objeto a representar, centrándose en los aspectos esenciales y evitando los detalles innecesarios. En POO, las clases son formas de abstracción que permiten modelar el mundo real de manera más manejable en el código. Por ejemplo: para abstraer el objeto Persona, nos debemos hacer preguntas del tipo... ¿Qué datos de una persona necesitamos almacenar en este proyecto? ¿Qué comportamientos de la Persona queremos simular?
- **Encapsulamiento (y ocultación):** una clase se compone tanto de variables (atributos) como de funciones y procedimientos (métodos). Agrupar esos elementos en un objeto es el encapsulamiento. De hecho no se pueden definir variables (ni funciones) fuera de una clase

(es decir no hay variables globales). Hay una zona oculta al definir la clase (zona privada) que sólo es utilizada por esa clase y por alguna clase relacionada, y generalmente son sus atributos; por otra parte, hay una zona pública (llamada también interfaz de la clase) que puede ser utilizada por cualquier parte del código, y es la única que nos permite interactuar con ese objeto evitando que su información interna se vea expuesta.

- **Herencia:** una clase puede heredar propiedades (atributos y métodos) de otra, especializando clases y creando una jerarquía de forma similar a cómo se realiza en el modelo Entidad-Relación (diseño de bases de datos). Mediante esta característica los objetos se pueden crear basándose en otra clase. Entre muchas ventajas, se puede destacar la reutilización de código.
- **Polimorfismo:** cada método de una clase puede tener varias definiciones distintas. En el caso del juego del parchís: `partida.empezar(4)` sería un método que empezaría una partida para cuatro jugadores, `partida.empezar(rojo, azul)` empezaría una partida de dos jugadores para los colores rojo y azul; estas son dos formas distintas de emplear el método `empezar`, que es polimórfico (tiene varias formas). También podríamos encontrar que el método `“empezar( )”` tuviera un comportamiento diferente si, en vez de invocarlo en una partida, lo invocáramos en otro tipo de objeto.

2. FUNDAMENTOS DE LAS CLASES

Una clase sirve para describir un conjunto de objetos que tienen características similares (atributos) y un comportamiento común (métodos), a modo de “plantilla” o “molde”. Las definiciones comunes (nombre de la clase, los nombres de los atributos y los métodos) se almacenan una única vez en cada clase, independientemente de cuántos objetos de esa clase estén presentes en el sistema. A partir de la clase (plantilla) se pueden crear objetos. Es decir antes de poder utilizar un objeto se debe definir la clase general a la que pertenece, esa definición incluye:

- **Atributos:** las variables miembro de la clase. Su acceso puede ser *public* (accesibles desde otra clase), *private* (sólo accesibles por código de su propia clase) o *protected* (accesibles por las subclases que hereden de ella). Los atributos representarán el estado e identidad.
- **Métodos:** las funciones miembro de la clase. Son las acciones u operaciones que puede realizar la clase si se le solicita. Al igual que los atributos, su acceso puede ser *public*, *private* o *protected*. Los métodos representarán su comportamiento.

En notación UML¹ la **estructura básica de una clase** se define así:

Nombre de la clase
Atributos
Métodos

1 [Unified Modeling Language](#)

Y la sintaxis para crear una clase en Java es la siguiente:

```
[acceso] class nombreDeClase {  
    [acceso] [static] tipo atributo1;  
    [acceso] [static] tipo atributo2;  
    //...más atributos...  
  
    [acceso] [static] tipo método1(listaDeParametros) {  
        //...código del método...  
    }  
    [acceso] [static] tipo método2(listaDeParametros) {  
        //...código del método...  
    }  
    //... más métodos...  
}
```

Veamos un ejemplo práctico: si queremos gestionar el conjunto de “Personas” en una aplicación, necesitaremos crear la clase “Persona”; ésta contiene 2 atributos (variables) para almacenar datos sobre las personas (nombre y edad), además de varios métodos (funciones) que hacen cosas con dichos datos.

```
public class Persona {  
    String nombre;  
    int edad;  
  
    // Establece o modifica el nombre de la persona  
    void setNombre(String n) {  
        nombre = n;  
    }  
  
    // Establece o modifica la edad de la persona  
    void setEdad(int e) {  
        edad = e;  
    }  
  
    // Devuelve el nombre de la persona  
    String getNombre() {  
        return nombre;  
    }  
  
    // Devuelve la edad de la persona  
    int getEdad() {  
        return edad;  
    }  
  
    // Muestra su nombre por pantalla  
    void imprimeNombre() {  
        System.out.println("El nombre es: " + nombre);  
    }  
  
    // Devuelve true si es mayor de edad, false en caso contrario  
    boolean esMayorEdad() {  
        return (edad >= 18);  
    }  
}
```

Hay que tener en cuenta que **la clase Persona nos servirá para crear tantos objetos Persona como necesitemos, cada uno con su nombre y edad**. Usar la clase para crear personas “concretas” y “diferentes” se denomina “**instanciar**”. Los métodos nos permitirán manipular los datos de cada objeto. Esto se explica con más detalle en el siguiente apartado. También es importante entender que **cada clase se crea en un archivo Java diferente** (con el mismo nombre de la clase), **y se utilizan fuera de la clase**. No obstante, veremos excepciones a esto.

Por ejemplo, podríamos tener un archivo Persona.java (con la clase Persona del ejemplo anterior) además de un archivo Programa.java (que solo tendrá la función principal `public static void main` a la que estamos acostumbrados). **Desde la función main de “Programa” podremos crear objetos de tipo Persona además de cualquier otro código que necesitemos**. Esto se verá en ejemplos posteriores de forma más detallada.

Los atributos y métodos de una clase se llaman *miembros de una clase*. Los atributos (variables) de una clase se llaman *variables de instancia* porque cada instancia de la clase (es decir, cada objeto de la clase), contiene sus propias variable atributo. Por lo tanto, los datos de cada objeto son individuales e independientes de los demás objetos generados. La palabra opcional *static* sirve para hacer que el método o el atributo a la que precede se pueda *utilizar de manera genérica*² (más adelante se hablará de clases genéricas), los atributos y métodos así definidos se llaman *atributos de clase y métodos de clase* respectivamente.

2.1 Visibilidad de los miembros de una clase

Los miembros de una clase (atributos y métodos, es decir, sus variables y funciones) pueden definirse como públicos, privados o protegidos; lo cual significa:

- **public**: se puede utilizar desde cualquier clase (acceso directo desde fuera de la clase).
- **private**: sólo puede utilizarlo la propia clase (métodos internos, nunca desde fuera).
- **protected**: puede utilizarlo la propia clase y también las subclases heredadas (por ahora no la utilizaremos, la herencia de clases se verá más adelante).

Llamaremos *interfaz* a los miembros de una clase (atributos y métodos) que están declarados como **public**, porque son los que permiten interactuar con la clase desde fuera de ella. Los principios de la programación orientada a objetos dicen que para mantener el **encapsulamiento** en los objetos debemos aplicar el especificativo **public** a las funciones miembro que formen la interfaz pública, y denegar el acceso a los datos miembro usados por esas funciones (variables de clase) mediante el especificador **private**.

⚡ Ojo: en un **paquete**, que es un agrupamiento lógico de clases en un mismo directorio, los atributos y los métodos de estas clases son **públicos** por defecto para el resto de clases existentes en el mismo paquete, y **privados** para cualquier clase que se encuentre fuera (a no ser que se especifique lo contrario).

² Es decir, se puede hacer uso de una llamada estática sin necesidad de crear una instancia para acceder, por ejemplo, a algún método de la clase. Esto ya lo hemos visto anteriormente cuando creábamos “funciones”...

Cuando un paquete no está definido, se dice que la clase pertenece al paquete por defecto, por lo tanto se aplica automáticamente el calificador *public* al resto de las clases cuyos ficheros se encuentren en el mismo directorio.

(Ver [ejemplo 3](#)).

2.2 Métodos

Se llama método a una función de una clase. Su sintaxis general es la misma que vimos para las funciones en general, y es la siguiente:

```
tipoDevuelto nombreMétodo(parámetros) {  
    // cuerpo del método  
}
```

Donde:

- el **tipo devuelto** especifica el tipo de datos que devuelve el método. Puede tratarse de cualquier tipo válido, incluyendo los tipos de clases que crea el programador. Si el método no debe devolver nada, el tipo devuelto especificado será *void*.
- el **nombre** del método, que puede ser cualquier identificador válido diferente a aquellos ya usados por otros elementos del programa.
- la **lista de parámetros** es una secuencia de pares de tipo e identificador separados por comas. Los parámetros son variables que reciben el valor de los argumentos que se pasa al método cuando se llama. Si el método no tiene parámetros, la lista de parámetros estará vacía.

Como en cualquier otra función, los métodos realizan la devolución de valores con la sentencia **return** a no ser que el tipo devuelto se defina como *void*:

Aunque sería perfectamente válido crear una clase que únicamente contenga datos, esto raramente ocurre y no se recomienda. Las buenas prácticas especifican la conveniencia de crear métodos para acceder a los atributos de la clase, y nunca acceder directamente a ellos.

Además de definir métodos que proporcionen acceso a los datos (lo cual ayuda a ocultar o abstraer la estructura interna de los datos) pueden definirse métodos para ser utilizados internamente por la clase para otras actividades.

2.3 Constructores

Puede resultar una tarea bastante pesada inicializar todas las variables de una clase cada vez que se crea una instancia. Incluso cuando se añaden funciones adecuadas, es más sencillo y preciso realizar todas las inicializaciones cuando el objeto se crea por primera vez. Como el proceso de inicialización es tan común, Java permite que los objetos se inicialicen en el momento en que se crean. Esta inicialización automática se lleva a cabo mediante un método que llamaremos constructor, el cual tiene el mismo nombre que la clase.

 Un **constructor** es un método especial que no devuelve nunca un valor, siempre devuelve una referencia a una instancia de la clase y es llamado automáticamente al crear un objeto de una clase, es decir al usar la instrucción **new**.

En general, cuando se crea una instancia de una clase no siempre se desea pasar los parámetros de inicialización al construirla, por ello existe un tipo especial de constructores que son los llamados **constructores por defecto**. Estos constructores **no llevan parámetros asociados**, e **inician los datos** asignándoles valores por defecto.

Cuando no se llama a un constructor de forma explícita, Java crea un constructor por defecto y lo llama cuando se crea un objeto. Este constructor por defecto asigna a las variables de instancia un valor inicial igual a **cero a las variables numéricas** y **null** a todas las referencias a objetos. Una vez se defina un constructor para la clase, se deja de utilizar el constructor por defecto.

En unidades posteriores veremos que una clase puede tener varios constructores.

 Existe la referencia **this**, que apunta a la instancia que llama al método, y siempre está accesible. Puede ser utilizada por las funciones miembro para acceder a los datos del objeto.

(Ver [ejemplo 4](#)).

3. OBJETOS

3.1 Instanciación (creación) de un objeto

Cuando creamos una clase estamos definiendo un nuevo tipo de datos que puede utilizarse para instanciar (crear) objetos de esa clase. La instanciación se hace de la siguiente manera:

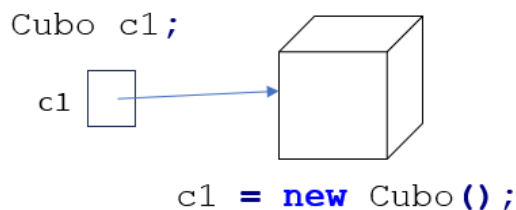
- En primer lugar, hay que **declarar una variable del tipo de la clase**.
- En segundo lugar, se necesitará una **copia física del objeto (reservada en memoria) y asignarla a esa variable**. Esto se hace utilizando el operador **new** que asigna dinámicamente (es decir, en tiempo de ejecución) memoria a un objeto y devuelve una referencia. Esta referencia se almacena en una variable (igual que hacemos con los arrays...)

Por ejemplo, suponiendo que ya hubiéramos definido previamente la clase Cubo, creamos uno:

```
Cubo c1;           // Crea una variable referencia llamada c1 de tipo Cubo
c1 = new Cubo();   // Crea en memoria un objeto "Cubo" y lo asigna a c1
```

También puede hacerse todo en una sola línea de código:

```
Cubo c1 = new Cubo();
```

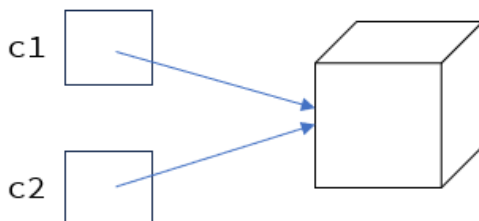


3.2 Asignación de variables de referencia a objetos

Las variables de referencia a objetos permiten acceder al objeto (son una referencia al objeto, no el objeto en sí). Por ello, actúan de forma diferente a lo que cabría esperar cuando tiene lugar una asignación. Por ejemplo, ¿qué hace el siguiente fragmento de código?

```
Cubo c1 = new Cubo();
Cubo c2 = c1;
```

Podríamos pensar que a c2 se le asigna una copia del objeto c1, pero **no es así**. Lo que sucede es que la referencia al cubo "c1" se copia en "c2", por lo que c2 permitirá acceder al mismo objeto referenciado por c1. Por lo tanto cualquier cambio que se haga al objeto referenciado a través de c2 afectará al objeto al que referencia c1, ya que es el mismo realmente. Es decir, no tenemos 2 cubos, sólo tenemos 1 cubo pero podemos acceder a él a través de 2 variables.



3.3 El operador punto (.)

Las variables y los métodos que pertenecen a una clase pueden ser accedidos mediante el operador punto (.), para ello debes:

1. Primero identificar la instancia (objeto concreto) que buscas, y luego,
2. Utiliza el operador de punto (.) para hacer referencia a la variable o método deseado.

Por ejemplo, supongamos que tenemos una clase llamada “Circulo”, con 2 atributos (radio y color) y 2 métodos (getRadio() y getArea()). Hemos creado 3 instancias de la clase “Circulo” (o sea, 3 círculos diferentes), llamados c1, c2 y c3. Para invocar el método getArea(), primero debes saber sobre cuál de los 3 objetos quieres invocarlo, por ejemplo sobre c2; luego, usaríamos el operador de punto (.) así:

```
c2.getArea();
```

Lo que estamos haciendo, literalmente, es pedirle al círculo “c2” que nos diga cuánto vale su área. Veamos otro ejemplo:

```
// Declarar y construir las instancias c1 y c2 de la clase Circulo
Circulo c1 = new Circulo();
Circulo c2 = new Circulo();

// Invocar métodos miembro para la instancia c1
System.out.println(c1.getArea());
System.out.println(c1.getRadio());

// Accedemos a variables de la instancia c2 a través del operador punto
c2.radio = 5;
c2.color = "azul";
```

Atención, esto NO debe hacerse. Dado que los atributos deben especificarse como “private”, las últimas líneas deberían sustituirse por algo como esto:

```
c2.setRadio(5);
c2.setColor("azul");
```

4. CONSTANTES DE CLASE Y DE OBJETO

Los miembros **constantes** se definen en Java a través de la palabra reservada ***final***, mientras que los miembros de clase se definen mediante la palabra reservada ***static***. De esta manera, si consideramos que los miembros son atributos, tenemos cuatro posibles combinaciones en Java para indicar si un atributo es constante:

- Atributo ***static***: toman valores comunes a todos los objetos existentes y potencialmente variables. Pueden utilizarse aunque no exista ningún objeto instanciado.
- Atributo ***final***: son valores constantes, pero potencialmente distintos en cada una de las instancias. Su valor se inicializa en la fase de construcción del objeto y no pueden ser modificados durante el tiempo de vida de éste.
- Atributos ***static final***: combinan las características de *static* y *final*.
- Resto de atributos (sin *static* ni *final*): atributos variables y diferentes para cada objeto.

⚡ Los **atributos static** de la clase deben ser **limitados**, ya que pueden dar lugar a errores de muy difícil depuración, además de ir contra el concepto de la programación orientada a objetos, y más concretamente del encapsulamiento.

Las constantes de objeto se definen mediante la palabra reservada ***final***. Se trata de atributos que toman el valor en el constructor del objeto, un valor que no puede ser modificado en el resto del programa. De esta manera, a cada objeto corresponde un atributo de ese tipo, pero invariable para él. Este tipo de atributos sirve para identificar cada objeto de forma única.

Ver [ejemplo 5](#).

5. ARRAYS DE OBJETOS

Hasta ahora, habíamos empleado arrays para almacenar tipos de datos primitivos (int, float, char...) pero también podemos almacenar otros tipos de datos, como los creados por nosotros, es decir, clases de objetos. En Java, la sintaxis para gestionar arrays de objetos es igual que para los arrays de tipos primitivos. Veamos cómo definir un **array de objetos**:

```
NombreClase[] objetos = new NombreClase[N];
```

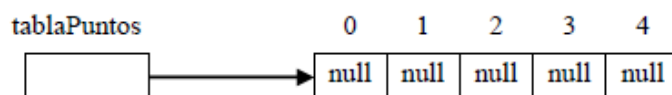
Hay que tener en cuenta que, cuando se crea el array, el compilador genera una referencia para cada uno de los elementos que lo integran, aunque no existan todavía las instancias de ninguno de ellos.

⚡ Para crear cada uno de los objetos a almacenar, hay que **llamar** explícitamente al **constructor por cada elemento creado**:

```
objetos[i] = new NombreClase();
```

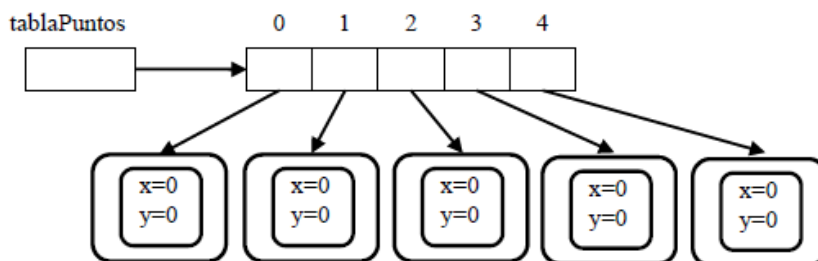
Veamos un ejemplo basándonos en el ejercicio propuesto con la clase “Punto”. Si ejecutamos esta instrucción obtendremos la estructura de la imagen:

```
Punto[] tablaPuntos = new Punto[5];
```



Ahora tendríamos que crear cada uno de los puntos que queremos almacenar:

```
for (int i = 0; i < 5; i++)  
    tablaPuntos[i] = new Punto();
```

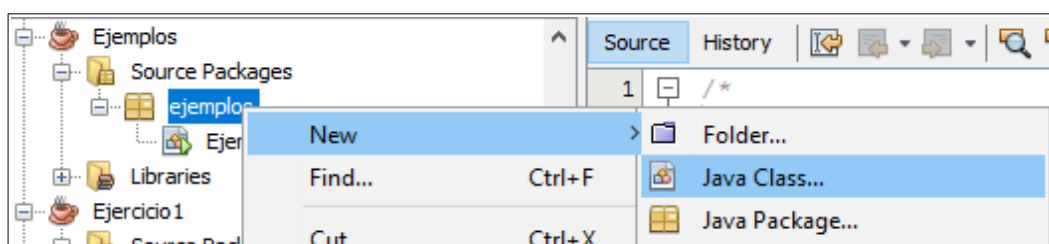


(Ver otro caso en: [ejemplo 6](#)).

6. EJEMPLOS

6.1 Ejemplo 1

En este ejemplo vamos a implementar la clase *Articulos*. Esta clase representará cada objeto (para un videoclub) con los siguientes atributos: *cod*, *titulo*, *formato* y *precio_alquiler*. También define tres métodos que permiten calcular, respectivamente, el precio de alquiler de un día, de dos días y una semana. Lo primero que necesitaremos, y esto lo haremos por cada clase que necesitemos en todos los ejemplos, será crear una nueva clase en Java. Para ello pincharemos con el botón derecho sobre el paquete donde vayamos a tener las clases y luego en *NEW > Java Class*. Crearemos una clase llamada *Articulo*.



```
12 public class Articulo {
13     // Atributos de la clase
14     String cod;
15     String titulo;
16     String formato;
17     float precio_alquiler;
18
19     // Métodos de la clase
20     float precio1() {
21         return (precio_alquiler);
22     }
23
24     float precio2() {
25         float precio_total;
26
27         precio_total = precio_alquiler * 1.80f;
28
29         return (precio_total);
30     }
31
32     float precio_semana() {
33         float precio_total;
34
35         precio_total = precio_alquiler * 5;
36
37         return (precio_total);
38     }
39 }
```

El método *precio1* devuelve el valor del precio de alquiler del artículo. El método *precio2* calcula el precio de alquiler de dos días haciendo un descuento del 20% (por eso se multiplica por 1,8). Por último el método *precio_semana* calcula el precio de una semana multiplicando por 5 el precio de alquiler.

Vamos a crear una clase nueva llamada Ejemplos, con la función public static void main, donde vamos a instanciar (crear) objetos de la clase Artículo y utilizarlos.

Con el operador *new* crearemos una instancia de la clase *Artículo*. Como por el momento la clase no tiene constructores, el sistema invocará al constructor por defecto de la clase. RECUERDA: cuando instanciamos una clase estamos creando un objeto concreto de esa clase.

```
12 public class Ejemplos {
13
14     public static void main(String[] args) {
15         // Creamos dos artículos
16         Artículo articulo1 = new Artículo();
17         Artículo articulo2 = new Artículo();
18
19         // Le damos valores a sus atributos
20         articulo1.cod = "001";
21         articulo1.titulo = "Titulo1";
22         articulo1.formato = "DVD";
23         articulo1.precio_alquiler = 2.50f;
24
25         articulo2.cod = "002";
26         articulo2.titulo = "Titulo2";
27         articulo2.formato = "DVD";
28         articulo2.precio_alquiler = 3;
29
30         // Utilizamos sus métodos
31         System.out.println("Alquiler Art. " + articulo1.cod + ", 1 dia:" + articulo1.precio1());
32         System.out.println("Alquiler Art. " + articulo1.cod + ", 2 dias:" + articulo1.precio2());
33         System.out.println("Alquiler Art. " + articulo1.cod + ", 1 semana:" + articulo1.precio_semana());
34         System.out.println("Alquiler Art. " + articulo2.cod + ", 1 dia:" + articulo2.precio1());
35         System.out.println("Alquiler Art. " + articulo2.cod + ", 2 dias:" + articulo2.precio2());
36         System.out.println("Alquiler Art. " + articulo2.cod + ", 1 semana:" + articulo2.precio_semana());
37     }
38 }
39 }
```

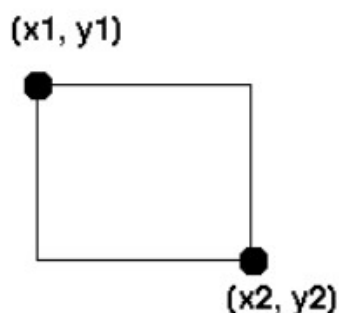
La salida de ese programa es:

```
run:
Alquiler Art. 001, 1 dia:2.5
Alquiler Art. 001, 2 dias:4.5
Alquiler Art. 001, 1 semana:12.5
Alquiler Art. 002, 1 dia:3.0
Alquiler Art. 002, 2 dias:5.3999996
Alquiler Art. 002, 1 semana:15.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

6.2 Ejemplo 2

En este ejemplo vamos a implementar la clase *Cuadrado*, que representa cuadrados mediante dos coordenadas 2D, y define tres métodos que permiten calcular, respectivamente, la diagonal, el perímetro y el área. Con esta clase podremos representar aspectos geométricos desde un punto de vista analítico.

El criterio de representación toma las coordenadas horizontales (x) crecientes de izquierda a derecha, y las verticales (y) crecientes de arriba abajo. Veamos una forma gráfica de entenderlo:



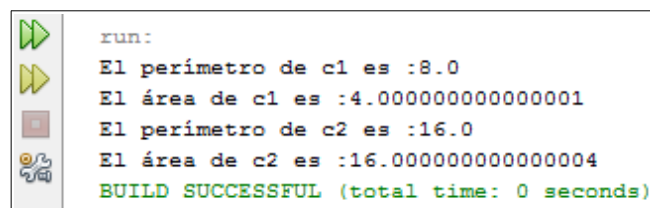
El código de Cuadrado.java sería:

```
12 public class Cuadrado {
13     // Atributos
14     double x1, y1, x2, y2;
15
16     // Métodos
17     double CalcularDiagonal()
18     {
19         return Math.sqrt((x2-x1)*(x2-x1)+(y2-y1)*(y2-y1));
20     }
21
22     double CalcularPerimetro() {
23         double diagonal = CalcularDiagonal();
24         double lado = diagonal/Math.sqrt(2);
25
26         return (4*lado);
27     }
28     double CalcularArea()
29     {
30         double diagonal =CalcularDiagonal();
31
32         return (0.5*diagonal*diagonal);
33     }
34 }
```

El código de Ejemplos.java sería:


```
12 public class Ejemplos {
13
14     public static void main(String[] args) {
15         // Creamos dos cuadrados
16         Cuadrado c1 = new Cuadrado();
17         Cuadrado c2 = new Cuadrado();
18
19         // Les damos valores
20         c1.x1=2; c1.y1=2; c1.x2=4; c1.y2=4;
21         c2.x1=1; c2.y1=1; c2.x2=5; c2.y2=5;
22
23         // Usamos sus métodos
24         System.out.println("El perímetro de c1 es :" + c1.CalcularPerimetro());
25         System.out.println("El área de c1 es :" + c1.CalcularArea());
26         System.out.println("El perímetro de c2 es :" + c2.CalcularPerimetro());
27         System.out.println("El área de c2 es :" + c2.CalcularArea());
28     }
29 }
```

Y la salida:



```
run:
El perímetro de c1 es :8.0
El área de c1 es :4.0000000000000001
El perímetro de c2 es :16.0
El área de c2 es :16.0000000000000004
BUILD SUCCESSFUL (total time: 0 seconds)
```

6.3 Ejemplo 3

En este ejemplo vamos a aplicar el principio de **encapsulamiento** haciendo *private* los atributos de la clase y *public* los métodos. Para ello vamos a modificar la clase Artículo del Ejemplo 1.

Pero ahora no podremos leer ni modificar los atributos de la clase desde fuera de ella (porque son *private*). Así que vamos a definir métodos que nos permitan hacerlo correctamente:

- Crearemos métodos de acceso *public* (uno por atributo) que nos devuelvan el valor de cada atributo. A esto se le llama métodos “*get*” o “*getters*” (del inglés tomar/coger).
- Del mismo modo, crearemos métodos que nos permitan modificar el valor de los atributos. A éstos se les llama métodos “*set*” o “*setters*” (del inglés establecer). En el ejemplo se llama *modificaValores* y permite cambiar todos los valores en una sola llamada.

El código de Artículo.java ahora sería:

```
12 public class Artículo {
13     // Atributos de la clase
14     private String cod;
15     private String titulo;
16     private String formato;
17     private float precio_alquiler;
18
19     // Métodos de la clase
20     public float precio1(){
21         return (getPrecio_alquiler());
22     }
23
24     public float precio2(){
25         float precio_total;
26
27         precio_total = getPrecio_alquiler() * 1.80f;
28
29         return (precio_total);
30     }
31
32     public float precio_semana(){
33         float precio_total;
34
35         precio_total = getPrecio_alquiler() * 5;
36
37         return (precio_total);
38     }
39
40     public void modificaValores(String cod_p, String titulo_p, String formato_p, float precio_p)
41     {
42         cod=cod_p;
43         titulo=titulo_p;
44         formato=formato_p;
45         precio_alquiler=precio_p;
46     }
47
48     public String getCod()
49     {
50         return cod;
51     }
52
53     public String getTitulo()
54     {
55         return titulo;
56     }
57
58     public String getFormato()
59     {
60         return formato;
61     }
62
63     public float getPrecio_alquiler()
64     {
65         return precio_alquiler;
66     }
67 }
```

El código de Ejemplos.java con el main sería:

```

12 public class Ejemplos {
13
14     public static void main(String[] args) {
15         // Creamos dos artículos
16         Artículo articulo1 = new Artículo();
17         Artículo articulo2 = new Artículo();
18
19         // Le damos valores a sus atributos
20         articulo1.modificaValores("001","Titulo1","DVD",2.5f);
21         articulo2.modificaValores("002","Titulo2","DVD",3f);
22
23         // Utilizamos sus métodos
24         System.out.println("Alquiler Art. " + articulo1.getCod() + ", 1 dia:" + articulo1.precio1());
25         System.out.println("Alquiler Art. " + articulo1.getCod() + ", 2 dias:" + articulo1.precio2());
26         System.out.println("Alquiler Art. " + articulo1.getCod() + ", 1 semana:" + articulo1.precio_semana());
27         System.out.println("Alquiler Art. " + articulo2.getCod() + ", 1 dia:" + articulo2.precio1());
28         System.out.println("Alquiler Art. " + articulo2.getCod() + ", 2 dias:" + articulo2.precio2());
29         System.out.println("Alquiler Art. " + articulo2.getCod() + ", 1 semana:" + articulo2.precio_semana());
30     }
31 }
32

```

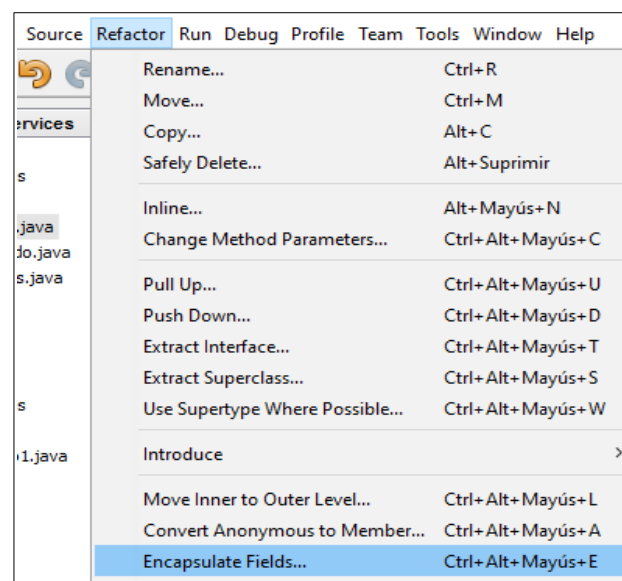
Y la salida:

```

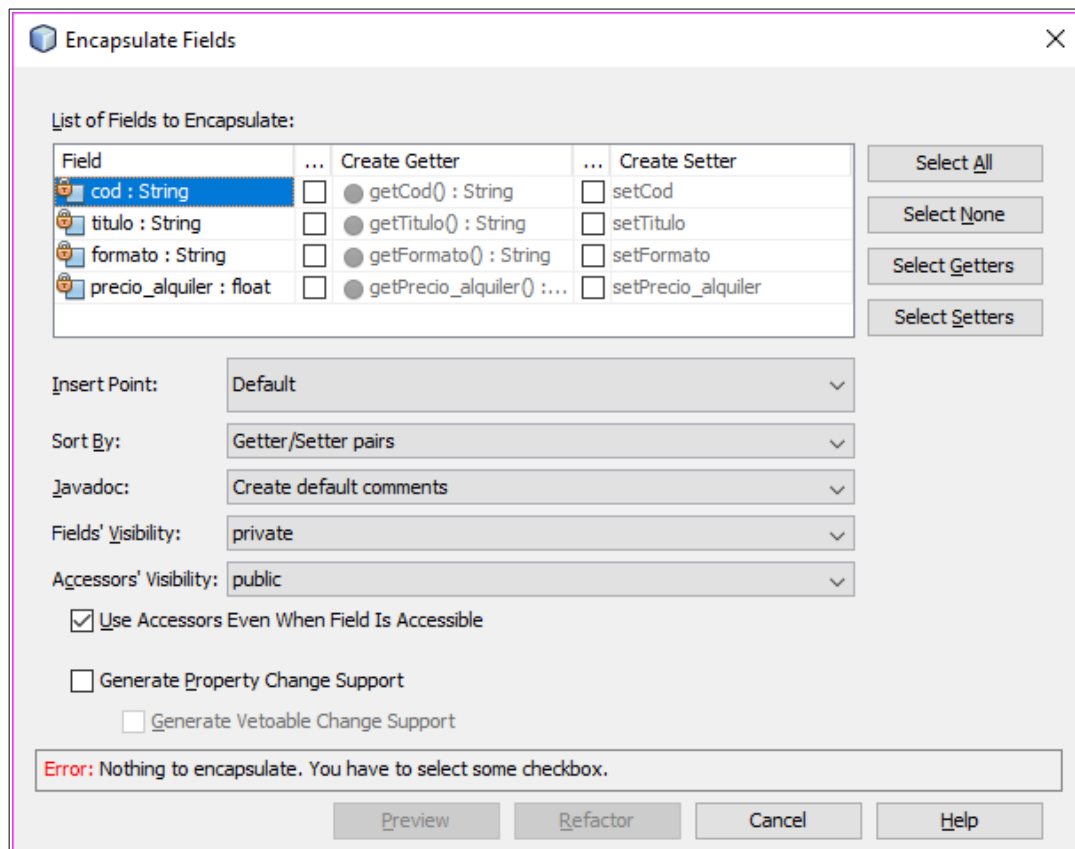
run:
Alquiler Art. 001, 1 dia:2.5
Alquiler Art. 001, 2 dias:4.5
Alquiler Art. 001, 1 semana:12.5
Alquiler Art. 002, 1 dia:3.0
Alquiler Art. 002, 2 dias:5.3999996
Alquiler Art. 002, 1 semana:15.0
BUILD SUCCESSFUL (total time: 0 seconds)

```

Podemos “ayudarnos” del IDE para crear los métodos *getters* y *setters* de forma automática, para ello usaremos “*Refactor*” de Netbeans (y Eclipse). Para ello pinchamos en el menú *Refactor* y luego en *Encapsulate Fields*.



Seleccionamos los *getters* y *setters* que queramos crear y pinchamos en *Refactor*.



Automáticamente aparecerán los métodos en el código.

6.4 Ejemplo 4

En este ejemplo vamos a ver cómo crear un constructor de clase y así evitar el uso del constructor por defecto. Para ello vamos a basarnos en la clase *Articulo* creada anteriormente y le vamos a añadir el constructor como vemos a continuación. El código de *Articulo.java* quedaría así:

```
12 public class Articulo {
13     // Atributos de la clase
14     private String cod;
15     private String titulo;
16     private String formato;
17     private float precio_alquiler;
18
19     // Constructor de la clase
20     public Articulo(String cod, String titulo, String formato, float precio_alquiler)
21     {
22         // Con 'this' estamos accediendo al objeto de la clase
23         this.cod = cod;
24         this.titulo = titulo;
25         this.formato = formato;
26         this.precio_alquiler = precio_alquiler;
27     }
28     // Métodos de la clase
29     public float precio1() {
30         return (getPrecio_alquiler());
31     }
```

Ahora en *Ejemplos.java* utilizamos el constructor (al instanciar el objeto con *new*). Esto permite instanciar el objeto y definir los valores de sus atributos en una sola instrucción.

```
12 public class Ejemplos {
13
14     public static void main(String[] args) {
15         // Creamos dos articulos
16         Articulo articulo1 = new Articulo("001", "Titulo1", "DVD", 2.5f);
17         Articulo articulo2 = new Articulo("002", "Titulo2", "DVD", 3f);
18
19         // Utilizamos sus métodos
20         System.out.println("Alquiler Art. " + articulo1.getCod() + ", 1 dia:" + articulo1.precio1());
21         System.out.println("Alquiler Art. " + articulo1.getCod() + ", 2 dias:" + articulo1.precio2());
22         System.out.println("Alquiler Art. " + articulo1.getCod() + ", 1 semana:" + articulo1.precio_semana());
23         System.out.println("Alquiler Art. " + articulo2.getCod() + ", 1 dia:" + articulo2.precio1());
24         System.out.println("Alquiler Art. " + articulo2.getCod() + ", 2 dias:" + articulo2.precio2());
25         System.out.println("Alquiler Art. " + articulo2.getCod() + ", 1 semana:" + articulo2.precio_semana());
26     }
27
28 }
```

Y la salida:

```
run:
Alquiler Art. 001, 1 dia:2.5
Alquiler Art. 001, 2 dias:4.5
Alquiler Art. 001, 1 semana:12.5
Alquiler Art. 002, 1 dia:3.0
Alquiler Art. 002, 2 dias:5.3999996
Alquiler Art. 002, 1 semana:15.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

6.5 Ejemplo 5

Para ilustrar el uso de los cualificadores **final** y **static** dentro de nuestra clase *Articulo*, vamos a definir tres atributos principales:

- La constante IVA.
- Un atributo compartido que contabilizará el número de instancias que se hayan definido hasta entonces.
- Un atributo constante String, distinto para cada objeto, que permita identificarlos.

Definiremos IVA como **static** y **final**, podremos acceder a su valor mediante la expresión *Articulo.IVA*, sin necesidad de haber creado ningún objeto de la clase *Articulo*:

```
public static final double IVA = 0.21;
```

El atributo privado con el número de instancia, en realidad, se trata de una variable global accesible para todos los objetos de tipo *Articulo*:

```
private static int numero = 0;
```

Y por último, el identificador constante de cada objeto se indica con un especificador **final** en su descripción:

```
final String identificador;
```

Se deja como ejercicio propuesto hacer estos cambios y probarlo.

6.6 Ejemplo 6

En este ejemplo vamos a crear un array (vector) que contenga 10 objetos de tipo *Articulo*. Los instanciamos todos con nombres genéricos, consecutivos y precios aleatorios.

Primero creamos un vector de Artículos llamado *misArticulos* (es un array de referencias a objetos *Articulo*). Luego instanciamos los 10 objetos, guardando cada uno de los objetos referenciados.

La clase *Articulo* no se vería modificada.

El código de Ejemplos quedaría así:

```

12 public class Ejemplos {
13
14     public static void main(String[] args) {
15
16         // Definimos el vector de Articulos de tamaño 10
17         Articulo[] misArticulos = new Articulo[10];
18         int cont;
19
20         // Para cada elemento del vector creamos el objeto Articulo
21         for (cont = 0; cont < misArticulos.length; cont++)
22             misArticulos[cont]=new Articulo("00"+cont,"Articulo "+cont,"DVD",(float)Math.random()*10);
23
24
25         // Recorremos el vector
26         for (cont = 0; cont < misArticulos.length; cont++)
27         {
28             System.out.println("Codigo Art. "+misArticulos[cont].getCod());
29             System.out.println("Alquiler 1 dia :"+misArticulos[cont].precio1());
30             System.out.println("Alquiler 2 dias :"+misArticulos[cont].precio2());
31             System.out.println("Alquiler 1 semana :"+misArticulos[cont].precio_semana());
32         }
33     }
34 }
35

```

La salida sería:

```

run:
Codigo Art. 000
Alquiler 1 dia :9.574945
Alquiler 2 dias :17.234901
Alquiler 1 semana :47.874725
Codigo Art. 001
Alquiler 1 dia :2.8240945
Alquiler 2 dias :5.08337
Alquiler 1 semana :14.120473
Codigo Art. 002
Alquiler 1 dia :6.450059
Alquiler 2 dias :11.6101055
Alquiler 1 semana :32.250294
Codigo Art. 003
Alquiler 1 dia :2.0261488
Alquiler 2 dias :3.6470678
Alquiler 1 semana :10.130744
Codigo Art. 004
Alquiler 1 dia :1.0488434
Alquiler 2 dias :1.887918
Alquiler 1 semana :5.244217
Codigo Art. 005
Alquiler 1 dia :9.384731
Alquiler 2 dias :16.892515
Alquiler 1 semana :46.923656
Codigo Art. 006
Alquiler 1 dia :8.714678
Alquiler 2 dias :15.6864195
Alquiler 1 semana :43.573387
Codigo Art. 007
Alquiler 1 dia :4.498719
Alquiler 2 dias :8.097694
Alquiler 1 semana :22.493595
Codigo Art. 008
Alquiler 1 dia :5.8765197
Alquiler 2 dias :10.577735
Alquiler 1 semana :29.382599
Codigo Art. 009
Alquiler 1 dia :1.7480422
Alquiler 2 dias :3.146476
Alquiler 1 semana :8.7402115
BUILD SUCCESSFUL (total time: 0 seconds)

```